

DETECTING PREY ANIMALS

Project Report

Hogeschool Inholland Amsterdam

Faculty of technology, design and computer science

Minor: Deep Learning

Kruijver, Ruben

Jurcanu, Dinu

Chuprin, Dmitry

Van Wingerden, Emily

Zinchenko, Taras

31-05-2026

Contents

1. List of Abbreviations 2
2. Context 3
3. Project Goal & Objectives 3
4. Main question & sub-questions 4
Theoretical framework 4
5. Methodology 5
Research Strategy 5
Research Design 5
6. Results & Analysis 7
Data 7
Frame extraction and preprocessing 8
Annotation 9
Prey species: Area 10
Prey species: Annotated 12
Base model 13
Evaluation metrics 13
Model training 14
Model architecture comparison 15
Extended YOLOv12 nano training 15
Current performance 17
Per class performance 19
Video Processing Pipeline 22
Audio-Guided Processing 23
7. In-The-Loop validation 25
8. Data Ethics Decision Aid 26
9. Conclusion 27
10. Discussion 28
11. References 29

1. List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface (a software interface)
COCO	Common Objects in Context (a large public dataset used for pre-training computer-vision models)
CSV	Comma-Separated Values (a plain-text spreadsheet format)
CVAT	Computer Vision Annotation Tool
DEDA	Data Ethics Decision Aid
F1 Score	Combined precision-and-recall score
GPU	Graphics Processing Unit (a specialized hardware for training neural network)
IoU	Intersection over Union
JSON	JavaScript Object Notation (a plain-text data format)
mAP	mean Average Precision
OpenCV	Open-source Computer-Vision software library
RT-DETR	Real-Time Detection Transformer (a transformer-based object detector)
STONE	Steenuilenoverleg Nederland (the Dutch little-owl research network)
YOLO	You Only Look Once (a family of object-detection models)

2. Context

A 50 yearlong analysis regarding the reproduction of the little owl concluded that owl nests located in areas (mostly) made up of clay ground came out worse than those in areas where the soil mainly consisted of sand. This also means that owlets, around the age of 10 days, brought up in an area where the soil was made up of sand are in better physical shape than those brought up on clay soil. The availability of nests in both types of areas does not explain the physical health of the owlets. What can thoroughly explain this is information regarding the feeding habits of these young [1].
In the Netherlands, owl populations are usually monitored with the use of nest boxes that have cameras equipped to them. These boxes are monitored throughout the breeding season. Recording these boxes provides footage of owls entering a nest with food/prey animals. This is valuable information, because it says something about the owl's feeding behavior, the area's availability of prey animals, and the area's ecosystem health.
The issue arises when the footage must be analyzed. Currently, thousands of videos have been watched and annotated manually to identify in which fragments certain prey animals are brought to the nest. What makes analyzing the footage more troublesome is that the footage is captured under difficult conditions, e.g., low light, infrared images (recorded with infrared light so the camera works in the dark, producing greyscale frames), motion blur and partially occluded prey (where the prey is partly hidden behind the owl's beak, feathers or feet).
This process must be automated with the use of computer vision [2] (the field of teaching computers to interpret images) and machine learning (a branch of artificial intelligence in which a computer learns patterns from examples instead from rules written by a programmer) [3]. The main benefit of this, which is also the desired situation, is that researchers would be able to focus on interpretation rather than manual data collection. A project has been set up to tackle this issue. It is called the **Automatic Owl Prey Detection** project, and it is carried out in collaboration with SteenuilenOverleg Nederland (**STONE**), the Dutch volunteer organization that monitors little-owl populations and DataLab, the data-science research group at Hogeschool Leiden. The project overall focusses on applying AI responsibly in an ecological research setting.

3. Project Goal & Objectives

The goal of the project is to create a solution based on artificial intelligence (AI) that aids researchers of STONE in their work and does not replace them. This is mainly done by significantly reducing the amount of manual work that is required to correctly annotate the frames with prey animals. The objective of the project is to create an AI-based solution that must be designed, implemented, evaluated and improved that can:

- Detect moments when prey is present in a video,
- Identify the type of prey,
- Assign confidence scores, expressing how certain the model is about each prediction,
- And allow human experts to validate or correct the results.

4. Main question & sub-questions

Main question:
What AI-based solution, including a deep learning model, can accurately classify the prey animals that owls bring into their nest?

Sub-questions:

1. What is the structure and composition of the dataset provided by DataLab, Hogeschool Leiden and STONE?
2. What actions are required to be performed to effectively utilize the data to train, validate, and test a model?
3. Which deep learning model achieves the best classification performance for identifying prey animals in frames of owls bringing prey into their nest?
4. What solution, likely in the form of an interface, can be developed so end-users can easily validate and correct the label given to a prey animal by the model?

Theoretical framework

Two computer-vision approaches could in principle answer the research question: image classification and object detection.

Image classification

Assigns a single label to an entire image. A classifier would predict, per frame, which prey species is present, but cannot indicate where in the frame the prey is or how many prey items appear. It is also poorly suited to frames that contain only the owl, only the nest box interior, or multiple prey simultaneously.

Object detection

Predicts a bounding box (a rectangle drawn around an object marking exactly where it appears in the image) and a class label (which category the object belongs to, for example owl or worm) for each object instance in the image. This matches the research context for several reasons. A single frame may contain the owl, the prey, and parts of the nest box at once, all of which need to be distinguished from one another. The researchers ultimately want a count of prey items per video, which requires localizing and de-duplicating detections across consecutive frames. Human validators also need to see where in the frame the model thinks the prey is, so they can quickly confirm or correct it.

Final choice

For these reasons, the project uses object detection, not classification.

5. Methodology

Research Strategy

The research strategy that applies to this project is both quantitative and qualitative. The latter is due to the amount of literature review required for this assignment. In addition to that, multiple designs must be made in iterations for the AI-based solution. The research is quantitative because existing data will be explored and preprocessed. In other words, the project works more with numbers and thus becomes more numerical and measurable. The focus is then on the 'what', so the research depth that applies here is descriptive, but also explorative. This is due to the project focusing on the associations and causations between certain components. This relates to what model might suit the use-case that is tackled by this project, which likely needs pre-processed input. As a result, multiple components are then present of which it must be known how they work together [4].

Research Design

Sub-question 1: Data Structure

It is imperative to know the usefulness of the data that has been provided to move forward with the project. The idea is that a deep learning model will be chosen and developed according to what the structure and composition is of the data and the specific use-case that is to be solved. The structure relates to how the data is stored, organized, arranged, and related. This explains if the data is structured (organized into clear tables and fields), semi-structured (partly organized, for example videos with some labels attached), or indexed (searchable by an identifier). The composition dives deeper into the data. The composition says something about the present attributes, ranges (or distinct) values, measurement levels, and data types. Python scripts and Jupyter Notebooks (an interactive coding environment used in data science) [5] will be used to load and explore the smaller portions of the data that the available hardware can handle.

Sub-question 2: Data Annotation

The sub-question explicitly utilizes the word "actions" to describe all the steps that are necessary to change the data in some way so that a neural network model accepts it. These "actions" are not the standard form of data pre-processing. With data pre-processing here it is meant that data is converted to a different type, normalized (rescaled to a standard range), binned (grouped into ranges), etc. It includes simple transformations that make the data more usable for analysis or simple (machine learning) models. The total number and specific actions differ per model.

For example, The YOLO model is an object detector that places bounding boxes on objects and labels to said boxes. To build such a model one needs training data in the form of the raw images and a file specifying the bounding boxes and their labels of each image [6]. To get the training data, one needs to annotate the images themselves and give the correct labels. The images can be annotated manually or by using third-party solutions like Roboflow (an online annotation platform) or CVAT (the Computer Vision Annotation Tool) [7], [8]. Other models might require totally different steps or annotation formats.

Sub-question 3: AI-based Solution

The model, or pipeline, that is to be developed for the client forms the most important challenge together with uncovering the usefulness of the data. It is known that the model must be a robust classifier (a model that reliably assigns the correct category to each input) that can be effectively used after the project has finished. In addition, object detection is required to exactly pinpoint in which frames prey animal are present. Already many metrics could be relevant when assessing such a big model. The question then is what model can do this? Existing models can be utilized with some modifications that are applied to them. A good example of this is or are variants of the YOLO model for object detection [6], but other and perhaps more suitable models exist (YOLO and the alternative architectures will be explained in the Base Model section).

First, literary study is required. One way in which this will be performed is reading the official documentation of existing models and useful architectures of neural networks. The other method in which adequate information will be acquired is by going over third-party articles containing information regarding the same neural networks. This will provide a list of candidate models with a variety of used parameters and layers and distinct layers.

Then, the candidate models will be built, fine-tuned (adjusted by training them further on the project-specific data after they have already been trained on a general dataset), trained, and tested. This can be the most time-consuming step, because neural networks typically need a substantial amount of training data compared to traditional machine learning models [9].

Finally, the model with the best performance metrics and simplest architecture is fully tested and deployed.

Sub-question 4: In-The-Loop validation

It is imperative that the model can be used by the client(s). Researchers at DataLab, Hogeschool Leiden do not necessarily need an interface to effectively work with the deployed model. However, the volunteers of STONE, who label the prey animal per location, need an interface to validate the outcome of the model. For example, if a prediction has a low confidence score (the model's own estimate of how certain it is about that prediction, on a scale from 0 to 1) as to what a certain prey animal is, then someone of STONE can verify or correct the applied label. The solution will be uncovered by frequently discussing and designing possible solutions that can be paired with other components of the project.

6. Results & Analysis

This chapter described the results achieved for each method used to answer the sub-questions. First, the results are given and then conclusions are drawn that answer the sub-questions. This is shown separately for each sub-question and in order of how they are written in chapter 3.

Data

The original dataset consisted of 252,189 infrared video recordings (.MP4 is a standard video file format) from owl nest boxes each stored in its own Azure Blob. (an entry in Azure Blob Storage, the Microsoft cloud-storage service used by DataLab to host the project's data). The videos have been recorded during the breeding season (April, May, June) of 2024 and 2025. The locations the data originates from and the number of videos per location are:

- Meddo: 62.109
- Randwijk: 59.423
- Haarle: 58.714
- Twisk: 53.204
- Vosseveld: 18.739

What sticks out is that Vosseveld has less video data than the other locations. Another clear way that the data can be distinguished is in camera angles. Each nest box has four camera channels (referred to as 'ch1' to 'ch4'), each recording from a different angle. Angle 'ch1' (**Top left**) showed the side of the entrance of the nestbox with angle 'ch2' (**Top right**) showing the opening of the entrance. These two angles (or channels) showed the best what type of prey owls would bring in. Angle 'ch3' (**Bottom left**) showed the inside of the box and angle 'ch4' (**Bottom** right) showed the outside of the box with the owlets. These last two camera angles do not easily show what prey owls bring in as can be guessed from the following image showing the camera angles [9].



Figure 1, Positions and view of the cameras around the nestboxes

Unfortunately, here is an imbalance of the data present as well. As shown in bullet points below, camera angle “Ch3” is overrepresented in videos compared to the other camera angles. This is the result of the owlets’ movement activating the motion sensor which triggers the camera to record.

- Ch1: 53,407
- Ch2: 44,118
- Ch3: 100,971
- Ch4: 53,693

The last aspect of the data that widely varies is the video duration. Not all videos are of the same length in seconds. The vast majority of the videos are between a few tenths of seconds and five minutes long. The outliers are the videos that are longer than ten minutes long with some being longer than an hour. This is probably caused by mosquitos activating the motion sensor of the camera which causes it to record, but it does not stop due to the constant presence of the mosquitos.

What is quite uniform are the hours of the day on which the recordings start. No clear spikes were noticed in the data meaning that there are enough samples present of owls bringing prey to their nest during the night and the day. Another aspect of the data that looked positive was the number of videos being uploaded per day. Each month of the breeding season captures roughly the same number of videos with the number of videos dropping at the end of June.

Frame extraction and preprocessing

Since the YOLO model (baseline model for this project, that is, the first version of the model that serves as a starting point for further improvement for this project) cannot be trained directly on full video files, the recordings first had to be converted into separate image frames (still images extracted from a video) with bounding-box annotations. Before frame extraction, the available video files were filtered using metadata (descriptive information about each video, such as its location, camera, and duration) created during the project to understand the data itself without downloading all the data.

DataLab Hogeschool Leiden provided our project group with extra information. That information came in the form of a CSV file (a plain-text spreadsheet format) containing information stating what prey animals can be found in which frames over the whole of 2024 data.
Unfortunately, this was missing for 2025. Only valid video files were kept from that year. Files with missing information, duplicate entries, and recordings shorter than five seconds were removed. This step helped prevent unusable videos from entering the frame extraction process. Frames were extracted with the open-source software library OpenCV (a computer-vision library for working with images and videos in code) [10]. The pipeline did not save every frame from each video. Instead, it checked every fourth frame to make the video extraction process faster.

Motion was measured by comparing the current frame with the previous frame after converting the image to grayscale (a black-and-white representation of the image) and applying a blur (a smoothing filter that removes tiny pixel-level differences so only meaningful motion remains). The sensitivity for this motion was set at 25, indicating that a

moderate amount of change must be present to consider the pixel “changed”. The minimum number of pixels that must be considered “changed” was set at 5,000. Frames with too little movement were discarded, because they usually showed an empty or unchanged nest box interior.

A sharpness filter was also applied before saving frames. The sharpness was calculated using the Laplacian variance (a number that measures how sharp an image is, where higher means sharper) [11] was set at 40. Frames below the sharpness threshold were removed, because blurry images are less useful for annotation and model training. When multiple useful frames were found within the same second, only the sharpest frame was kept. This reduced near-duplicate frames and improved the quality of the final image set.

The selected frames were saved as JPEG images because that saves more memory compared to storing them as PNG images. A manifest file (a small index file listing every saved frame together with where it came from) was created to keep track of the frame name, source video, location, camera, frame number and sharpness score. These frames were then used for annotation.

Annotation

The annotation was mainly done with Label Studio (an open-source tool for drawing bounding boxes and assigning labels to images) [12], where bounding boxes were drawn around the owl and prey animals and the correct class label was assigned. In addition to Label Studio, custom Python review tools were used to inspect, correct, and combine annotations into a master annotation file. This made it possible to review both manually labeled data in JSON and imported YOLO-format labels (a YOLO-specific format where each image has a matching .txt file listing the bounding boxes and their class labels) in one workflow.

Guidelines were followed during the annotation of images. The first was to include as many images as possible of owls carrying prey into their nest. The second was to remember to include enough annotations for each species. Not enough species means that the model cannot precisely detect it and too many annotations of a species is unnecessary. There was no threshold set for too many annotations, but the threshold for too few annotations was 50. The third rule that was followed was not to include human faces in the images with annotations. The fourth rule was to include images containing no annotations. The last rule was to delete duplicate images if they were recognized during the annotation process. Only frames originating from cameras angles “ch1” and “ch2” due to how easy those angles make it for the viewer to see what prey is being brought into the nest.

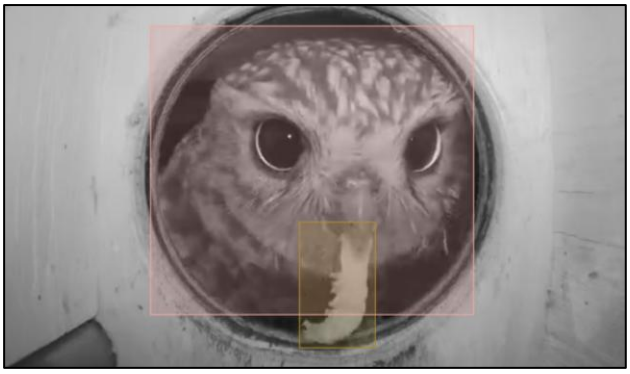


Figure 2, Annotation of owl entering its nest with a prey animal via camera angle “ch2”

Prey species: Area

It is imperative to know what to predict to develop any type of neural network. This can relate to classes of, for example, animal species, cars, weather status etc. What is known is what to classify/label, namely being species of prey that owls take into their nest to feed their young. However, the unknown is the number of species. A neural network (a kind of model loosely inspired by the human brain that learns by adjusting many connections during training) must know how many species, or classes (the categories the model learns to recognize), it needs to assign to the input. Luckily, this is research that has already been done in at least three locations in the Netherlands wherein nesting cameras are had been placed.

In a report published by STONE the topic of food availability was tackled for the little owl [13]. In more detail, statistics of owl young being brought up into areas where the ground was made up of sand looked better than owl young being brought up into areas where the ground was made up of clay. The places where these statistics originate from are *Vosseveld*, *Haarle*, and *Lichtenvoorde*. During the breeding season, the amount of prey taken into the nest was recorded, and the data was updated with each observation. All the mentioned places are in areas where the soil is mainly made up of sand.

Of all the locations, Vosseveld showed the best growth of the owl offspring. This could be because of their diet largely consisting of worms and mice. Haarle and Lichtenvoorde relatively share the same diet profile, but with a higher presence of other species like insects and amphibians. An explanation for this more varied diet is that these two places are closer to areas where the ground is made up of **river clay** (green). Lichtenvoorde is located near the border to the right of the Netherlands with Germany, as is visible on the map below. Lichtenvoorde is located further inland where the soil is made up of **sand** (yellow).

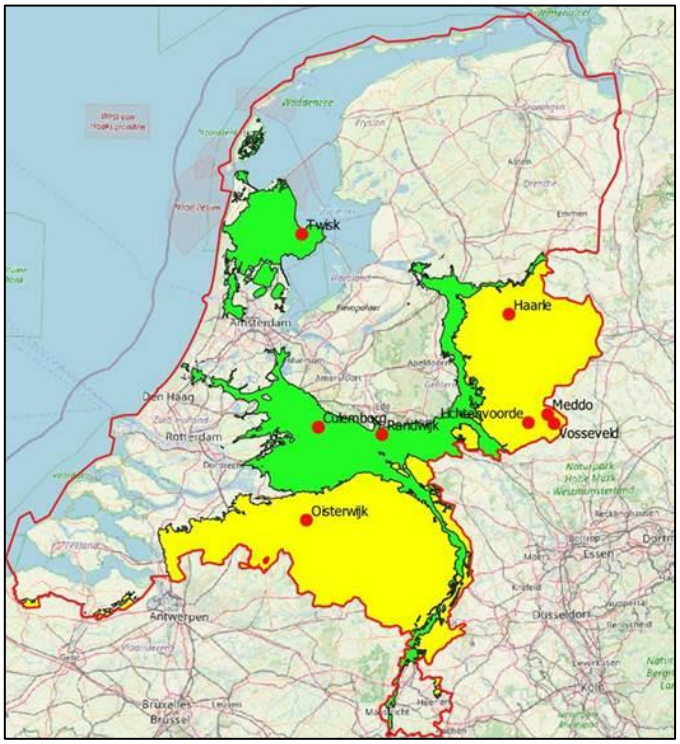


Figure 3, Map of locations with nestboxes and the soil type

The species presented in the table below are in Dutch, as was also the case in the original report of STONE, but the emphasis should be on the cells that share the same colour (except orange and white). The column to the **left** shows the minimum number of species that the AI-based model must detect. The column to the **right** shows the same column to the left but has been expanded to what it could maximumly be. The mice (Muis) species is only one global class to the left but five specific species to the right. The worm (Regenworm) Has been expanded into four cells according to its size and the cells in purple are expanded to distinct species from that look similar to each other. The difference in the number of species is **14** to **24**.

Classes minimum	Classes maximum
Steenuil	Steenuil
Muis	Bosmuis
Regenworm	Huismuis
Grote groene sabelsprinkhaan	Veldmuis
Spitsmuis	Rosse woelmuis
Kikker	Woelrat
Hagedis	Regenworm S
MeiKever	Regenworm M
Nachtvlinder	Regenworm L
Spin	Regenworm XL
Rups of larve	Grote groene sabelsprinkhaan
Insect	Spitsmuis
Vogel	Kikker
Haas*	Hagedis
	Watersalamander
	Meikever
	Kever
	Nachtvlinder
	Spin
	Rups
	Larve
	Insect
	Vogel
	Haas*

Figure 4, Table of minimum and maximum prey species brought into nestboxes

Frames of owls taking prey into their nest require to be annotated in the form of drawing bounding boxes and correctly labelling the species of prey that is present in a frame. More species means more work, at least one hundred annotated images per prey species [14], and more room for error. It is expected that the amount of time it takes to annotate the frames grows exponentially to the increase in species if the same quality in the annotating and labelling is to be upheld.

Prey species: Annotated

The previous two pages describe what species are present according to some data from three locations, but is it enough? The answer is yes, if all the pictures/frames containing owls and prey are considered. However, only frames showing the entrance of the nestbox ('ch2') have been annotated to train the model. This was done to provide the volunteers of STONE more ease in validating the classifications. Limiting the data to videos, and thus frames, originating from that camera angle results in less available data to use for training, tuning and testing models. The total dataset that has been annotated consists of labelled data from 2024, unlabeled data from 2025, and annotated data from the previous project group. The following graph portrays the amount of data that is then available for the species that are present.

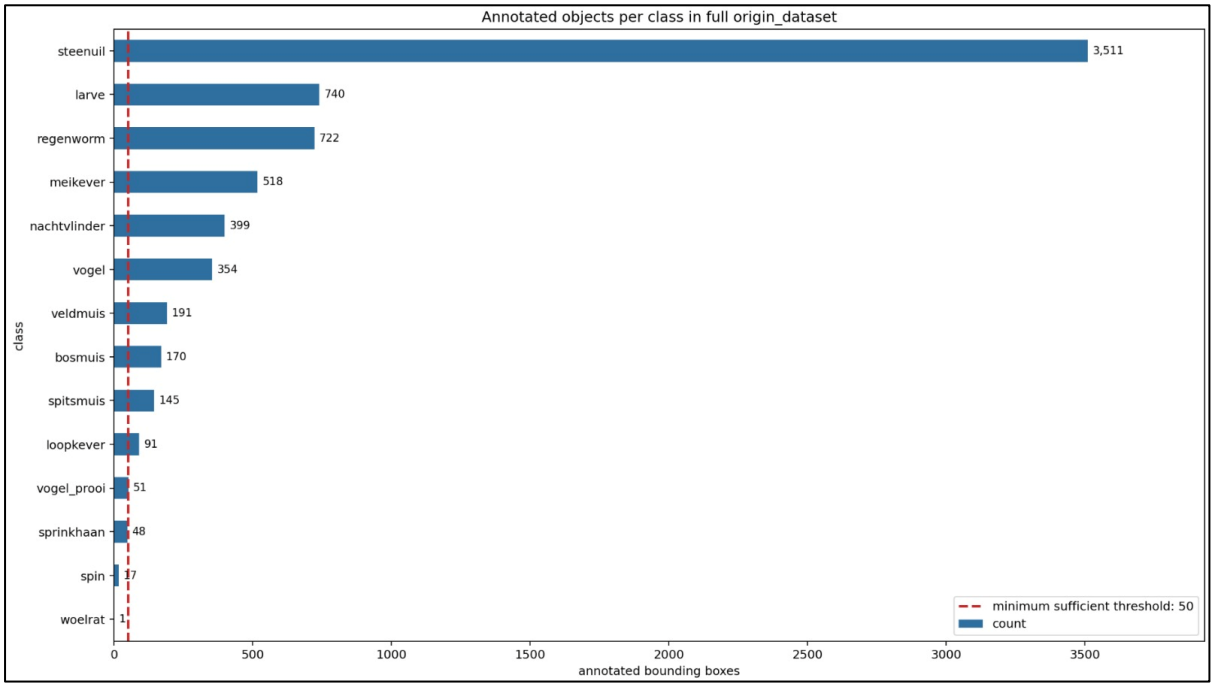


Figure 5, Graph showing amount of "ch2" frames annotated per species

Unfortunately, not enough data is present or annotated to include the species Spin and Woelrat. The goal was set at 50 annotations per species, but at least one hundred are needed to train a model to detect each species. Fortunately, there is enough data to train a model to precisely detect 12 species, including the little owl. The data was split into a train set of 3800 annotations and a test set of 579 annotations The number of annotated frames per location are:

- Vosseveld: 2339
- Hellendoorn: 1167
- Randwijk: 416
- Twisk: 252
- Haarle: 205

Base model

The YOLO ("You Only Look Once") family of models was selected for the baseline. YOLO is a single-stage detector that predicts bounding boxes and class probabilities (the model's estimated probability that each box belongs to each class) in one forward pass (a single run of the input image through the model) through the network, which makes it fast enough to process thousands of frames per video at reasonable cost. Compared with two-stage detectors such as Faster R-CNN, YOLO trades a small amount of localization precision (how tightly the bounding box hugs the real object) for a large gain in inference speed (how fast the model can produce predictions on new images), which matters for an applied research setting where the pipeline must scale to multiple nest boxes and full breeding seasons [15].

Concretely, the project uses **Ultralytics YOLOv12 nano [16]**. YOLOv12 was released in early 2025 and introduces an attention-centric design: where earlier YOLO versions relied purely on convolutional layers (the standard building blocks of computer-vision models, which detect visual patterns at increasing levels of abstraction), YOLOv12 incorporates an attention mechanism (a component that allows the model to focus on the most relevant parts of the image, similar to how a person focuses on what matters in a scene) aimed at improving the model's ability to focus on the relevant object in cluttered scenes which directly matches the challenge of locating a small prey item held in an owl's beak. The nano variant is the smallest of the available sizes (n, s, m, l, x) and was chosen because:

- It runs fast enough on commodity hardware to process full breeding-season footage without dedicated GPU infrastructure, keeping the pipeline deployable at STONE.
- It is small enough to train end-to-end (meaning every part of the model is trained from input to output rather than re-using fixed components) on the available annotated dataset without severe overfitting (a problem where the model memorizes the training images instead of learning patterns that work on new ones) [17].

Evaluation metrics

The model is evaluated using the standard object-detection metrics:

- Intersection over Union (IoU): the overlap between a predicted bounding box and the ground-truth bounding box (the correct bounding box drawn by a human annotator), expressed as the area of intersection divided by the area of union. A prediction is counted as a true positive (a correct detection) only if its IoU with the ground truth exceeds a threshold (typically 0.5) [18].
- Precision: fraction of predicted boxes that are correct. Low precision means the model produces many false positives, which directly increases the validator's workload [18]
- Recall: fraction of ground-truth prey instances that the model finds. Low recall means prey events are missed, which biases the ecological data [18].
- F1 score: the harmonic mean of precision and recall, used to summarize overall performance with a single number [19].
- mAP@0.5 : mean Average Precision at IoU threshold 0.5, averaged across classes. This is the headline metric reported per model [20].
- mAP@0.5:0.95 : mean Average Precision averaged across IoU thresholds from 0.5 to 0.95 in 0.05 steps. It is a stricter metric that rewards tight bounding-box localization [20].

For the use case, recall matters more than precision when prey is detected at all (a missed prey event is lost data forever), but per-class precision matters when distinguishing visually similar prey such as a different mouse species.

Model training

After frame extraction and annotation, the prepared dataset contained 4,379 annotated infrared frames. These frames contain labelled bounding boxes for the little owl and the prey animals visible in the selected nest-box frames. For model development, the annotated frames were divided into a training set and a held-out evaluation set. The training set was used to update the model weights (the internal numerical values the model adjusts during training so its predictions become more accurate [21]), while the held-out evaluation set was kept separate from the weight updates and used to measure the performance of the model. In this report, this held-out set is used as the final evaluation set for the current stage.

Dataset subset	Annotated frames	Proportion of frames
Training set	3,800	86.78%
Held-out evaluation set	579	13.22%
Total	4,379	100.00%

The held-out evaluation data contains the same main object-detection task as the training data: detecting the little owl and identifying visible prey animals. The distribution of the main classes remains suitable for evaluating the model, because the most frequently occurring prey categories are present in both subsets. The available data naturally contains more examples of the little owl and frequently observed prey animals than of rare prey species. This is expected in an ecological video dataset, because the owl is visible in many selected frames, while prey types occur at different frequencies.

The model was developed using frames from one selected camera channel. This channel was chosen because its cameras are directed towards the entrance of the nest box, where an owl carrying prey is most clearly visible. Since the same type of entrance-facing view is used across the selected locations, the model receives frames with a largely comparable perspective. This makes the selected channel especially suitable for the first object-detection model, because the model can focus on recognising the owl and prey rather than learning substantially different camera viewpoints.

We used:

- **Hardware:** Training was done on Kaggle cloud using dual NVIDIA T4 GPUs (~30gb of vram)
- **Training Time:** Complete training run of ~70-150 epochs (depending on the run) took us approximately 50 to 90 minutes to complete on the Kaggle GPU hardware.
- **Software:** Training was done using Jupyter Notebooks, relying on the ultralytics lib for the YOLOv12 nano architecture and OpenCV for handling image data, and PyTorch as backend.

Model architecture comparison

Three object-detection architectures were trained and compared under the same initial training budget of 50 epochs (one complete pass through the entire training dataset [22]): YOLOv12 nano, RT-DETR and YOLOv10 nano. This experiment made it possible to select the model architecture based on observed performance on the project dataset instead of only theoretical expectations.

Model architecture	Epochs	Precision	Recall	mAP@0.5	mAP@0.5:0.95
YOLOv12 nano	50	0.9954	0.7805	0.8810	0.5434
RT-DETR	50	0.8675	0.7983	0.8262	0.4307
YOLOv10 nano	50	0.8999	0.7073	0.7532	0.4636

YOLOv12 nano achieved the highest precision, mAP@0.5 and mAP@0.5:0.95 scores in the comparison. RT-DETR achieved a slightly higher recall, meaning that it detected a slightly larger proportion of the annotated objects. However, YOLOv12 nano provided the strongest overall object-detection result, especially through its higher mAP scores. These scores show that YOLOv12 combines correct detections with more accurate bounding-box localization.

This balance is important for the intended use of the model. In the planned human-in-the-loop workflow, the predictions made by the model will be presented to a user for validation. A model that detects objects accurately and places bounding boxes correctly can provide useful first suggestions and reduce the amount of manual correction required during the validation process.

Based on these results, YOLOv12 nano was selected for the extended training run. The nano variant is also appropriate for this project because it is lightweight. This is relevant for later applications to extracted video frames, where both detection quality and processing efficiency are important.

Extended YOLOv12 nano training

After the initial architecture comparison, YOLOv12 nano was trained for a longer run to further improve its detection performance. The recorded configuration of this experiment is shown below.

Training setting	Value used
Model architecture	YOLOv12 nano
Maximum configured epochs	300
Early stopping patience	30 epochs

Completed epochs	119
Best evaluation epoch	89
Batch size	128
Training image size	640 × 640 pixels
Configured initial learning rate	0.01
Optimiser setting	Automatic selection

The model was allowed to train for a maximum of 300 epochs, while early stopping was applied with a patience (number of epochs the training is allowed to continue without improvement before it stops [23]) value of 30 epochs. Early stopping means that training automatically finishes when the evaluation performance has not improved for a specified number of epochs. In this experiment, the best evaluation checkpoint was reached at epoch 89 and training stopped after epoch 119. This ensured that the most suitable checkpoint was retained without continuing the training process after its best observed evaluation performance had already been reached.

A batch size (the number of training images processed together in one training step) of 128 was used during the extended training run. This allowed the model to process a relatively large number of images in each training step and made efficient use of the available computing resources. Since this stage of the project mainly focused on selecting and improving the model architecture, the batch size was kept as a stable training setting rather than being varied in a separate experiment.

The model was trained with an image size of 640 × 640 pixels. This resolution preserves enough visual detail for object detection while keeping the computational requirements manageable.

The choice of image size was additionally evaluated after training through a grid search (systematically trying every combination of selected parameter values to find the best one [24]) on the held-out evaluation set. In this experiment, the trained model was tested at image resolutions (the number of pixels along each side of the image) of 640, 960 and 1280 pixels, combined with confidence thresholds of 0.15, 0.25, 0.35, and 0.50. The purpose of this comparison was to determine whether a larger image resolution during prediction would improve the detection of small prey animals.

The grid-search results showed that increasing the image size did not improve detection performance for the current trained model and dataset. The larger resolutions reduced prey recall and overall bounding-box recall, meaning that more relevant objects were missed during prediction. They also required considerably more processing time. At an image size of 640 pixels, the model processed approximately 10.40 images per second, while at 960 pixels the processing speed decreased to approximately 3.70 images per second.
Based on this experiment, an image size of 640 × 640 pixels was retained for the final prediction configuration. It provided the most suitable balance between detection quality and processing speed.

This is a useful practical result for the application of the model: although larger images contain more pixels, the evaluation experiment showed that they did not improve prey detection in the current pipeline and would make large-scale frame processing slower.

The configuration records an initial learning rate of 0.01 and an optimiser setting of auto. Automatic optimiser selection means that the training framework selects an appropriate optimization method for the configured model run, rather than requiring to manually choose and tune an optimiser before the model architecture has been established.

No separate custom augmentation pipeline was added beyond the training framework configuration. This means that the extended model result was established on image conditions that closely match the prepared infrared nest-box frames. Since the selected camera channel already provides a largely consistent entrance-facing view of the nest box, this setup allowed the model to learn directly from the type of footage it is expected to process.

Current performance

The initial architecture comparison showed that YOLOv12 nano was the most suitable candidate among the three tested models. The extended YOLOv12 nano training run then improved the results on the held-out evaluation set further. The best-performing checkpoint was reached at epoch 89, while the completed training process stopped at epoch 119 due to the early stopping configuration.

YOLOv12 nano evaluation result	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Initial comparison run, 50 epochs	0.9954	0.7805	0.8810	0.5434
Best extended-run checkpoint, epoch 89	0.9392	0.8675	0.9603	0.8009
Final completed epoch, epoch 119	0.9293	0.8665	0.9530	0.7794

The extended training run resulted in a substantial improvement in both reported mAP scores. During the initial 50-epoch comparison, YOLOv12 nano achieved an mAP@0.5 score of 0.88 and an mAP@0.5:0.95 score of 0.54. At the best checkpoint of the extended run, these scores increased to 0.96 and 0.80 respectively. This improvement shows that the longer training run helped the model to detect objects more effectively and to place bounding boxes more accurately.

The best checkpoint achieved a precision score of 0.94. Precision expresses the proportion of predicted detections that correspond to correctly annotated objects. In practical terms, this means that most bounding boxes proposed by the model in the held-out evaluation frames represent relevant detections.

The recall score of 0.87 indicates that the model successfully detected a large majority of the annotated objects in the held-out evaluation set. Recall is particularly relevant for this project, since missed prey detections can result in feeding events (moments when an owl delivers prey to the nest) not being included in later analysis. The improvement in recall compared with the initial comparison run shows that the extended YOLOv12 nano training run helped the model recognize more of the relevant owl and prey objects.

The model achieved a mAP@0.5 score of 0.96 at its best evaluation checkpoint. This indicates that the model was able to correctly identify and localize the annotated objects in most evaluation cases when using the standard Intersection over Union threshold of 0.50. The stricter mAP@0.5:0.95 score of 0.80 shows that the model also maintained good localization quality when evaluated across more demanding overlap thresholds.
This result is relevant for prey detection in nest-box footage. Prey animals can be small, partly hidden by the owl and difficult to distinguish in infrared images. The improvement in mAP@0.5:0.95 indicates that the trained model became more capable not only of identifying relevant objects, but also of placing bounding boxes accurately around the visible owl and prey items.

The result from epoch 89 is used as the best model checkpoint because it achieved the highest recorded evaluation performance. Although the model continued training until epoch 119, the metrics at the final epoch were slightly lower than those of the saved best checkpoint. Retaining the best checkpoint ensures that the selected model corresponds to the strongest observed result from the completed training run.
Overall, the completed experiments support the selection of YOLOv12 nano as the current model for the project. It achieved the strongest overall result in the initial architecture comparison and improved further during extended training. In combination with its practical processing speed at an image size of 640 × 640 pixels, the model provides an effective basis for automatically proposing owl and prey detections that can later be reviewed within the human-in-the-loop validation workflow.

To understand where the yolo model struggles, we looked at the confusion matrix and checked the exact frames where the model made mistakes. We found a few main issues:

Background Confusion: The model doesn't really get confused between different prey species. Instead, its biggest problem is mistaking static background parts of the box for prey.

Empty Beak False Detections: Sometimes, when an owl flies in with nothing in its beak, the model still thinks there is a prey animal there. It tends to hallucinate a prey detection around the beak area even when it is completely empty:

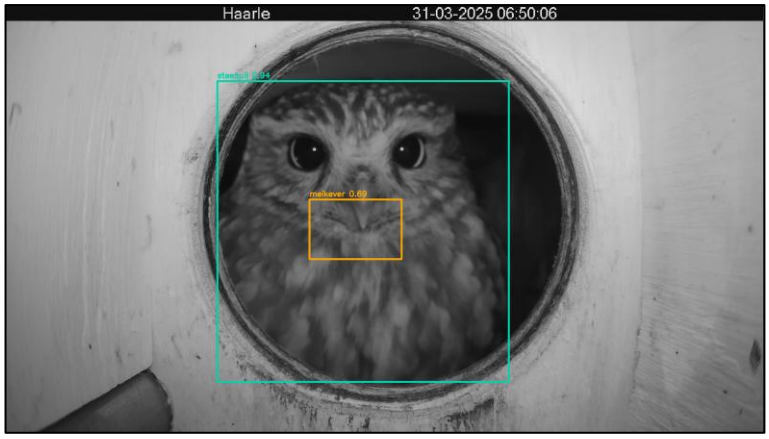


Figure 6, Detection model mistaking the beak of the owl for an insect

Nest Box Features: The model occasionally mistakes empty nest-box entrance holes, dark shadows, or the wood patterns on the walls for small mammals or insects.

Motion Blur: When an owl flies into the nestbox very fast at night, the infrared camera image gets blurry. This blur makes it hard for the model to see the clear shape of the prey, which leads to wrong predictions.

Per class performance

In addition to the overall evaluation metrics, the selected YOLOv12 nano model was evaluated per class at an IoU threshold of 0.50. This analysis gives a more detailed view of the model's behavior across the final set of classes used for evaluation. It is especially useful in this project because the dataset contains both frequent classes, such as steenuil, and less frequent prey classes.

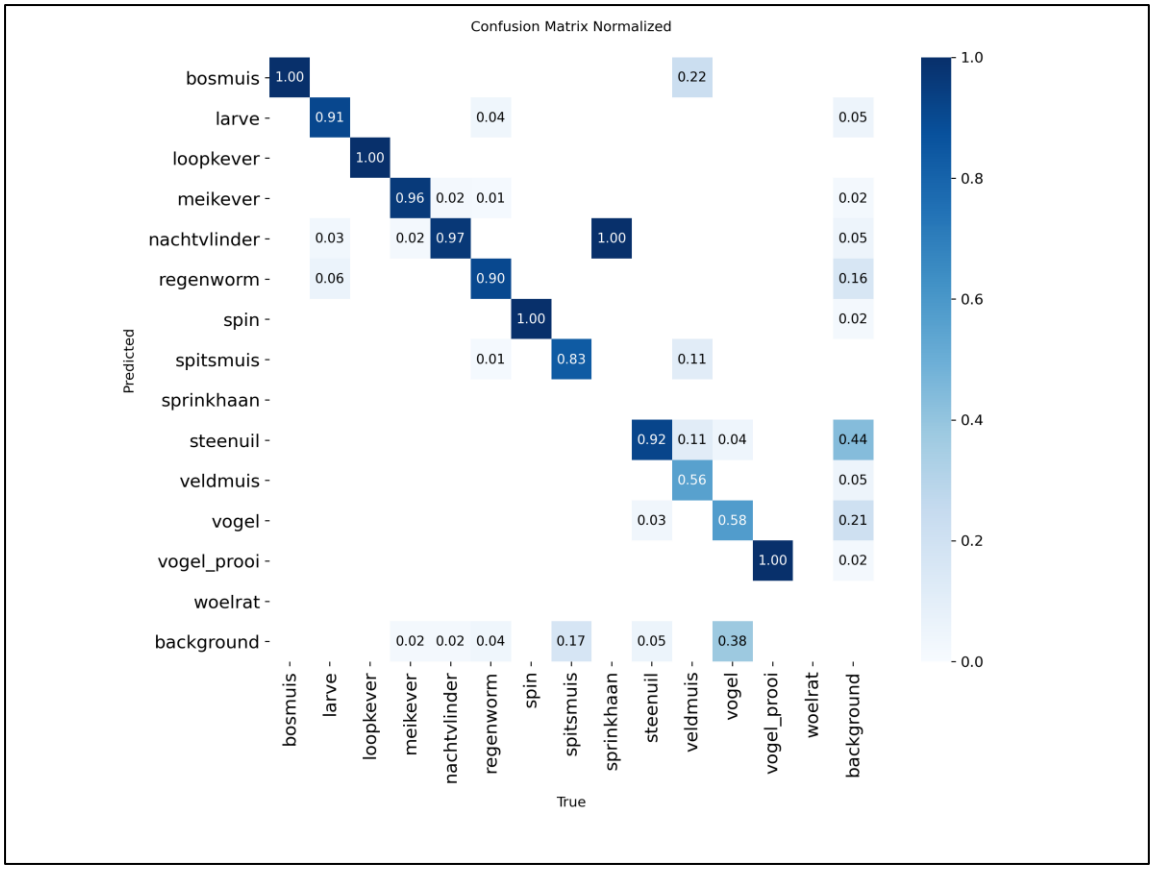


Figure 7, Confusion matrix of model's current performance

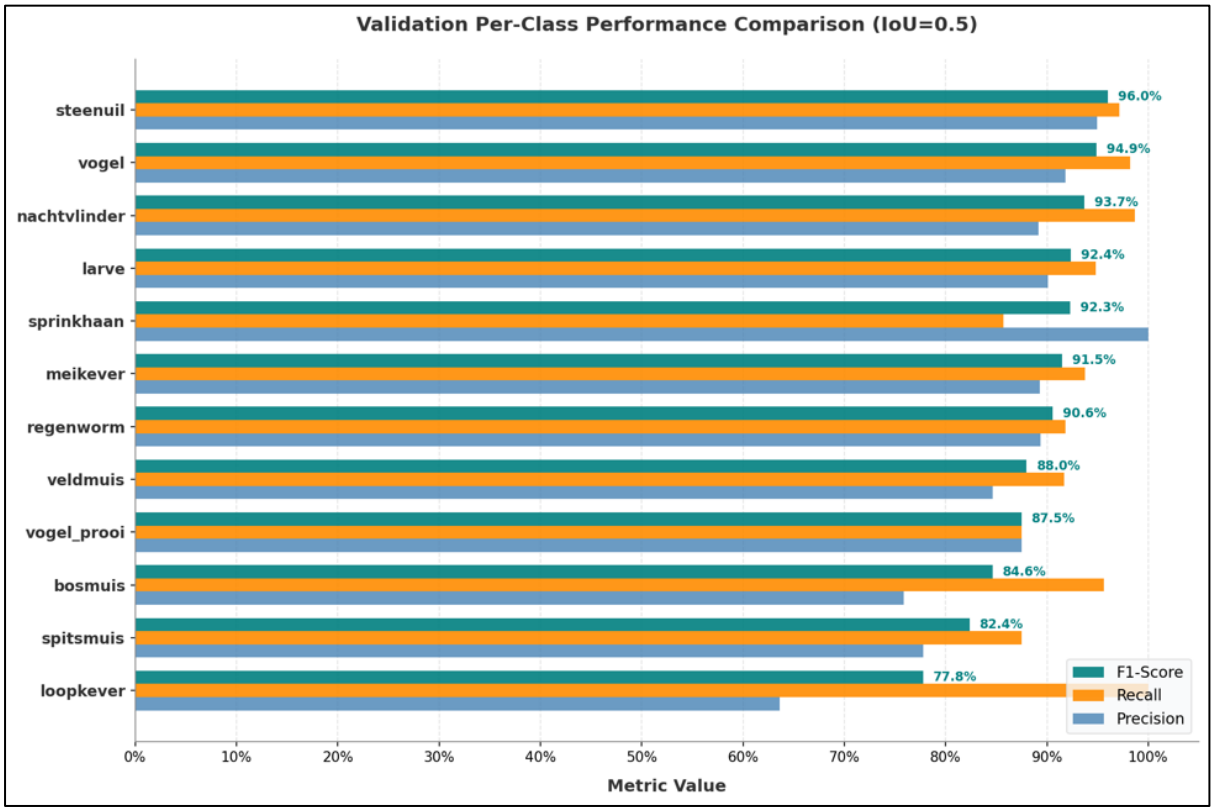


Figure 8, Performance values of the model for each class

Class	Ground-truth count	Precision	Recall	F1-score
steenuil	525	0.9497	0.9714	0.9605
vogel	57	0.9180	0.9825	0.9492
nachtvlinder	75	0.8916	0.9867	0.9367
larve	96	0.9010	0.9479	0.9239
sprinkhaan	7	1.0000	0.8571	0.9231
meikever	80	0.8929	0.9375	0.9146
regenworm	110	0.8938	0.9182	0.9058
veldmuis	24	0.8462	0.9167	0.8800
vogel_prooi	8	0.8750	0.8750	0.8750
bosmuis	23	0.7586	0.9565	0.8462
spitsmuis	16	0.7778	0.8750	0.8235
loopkever	7	0.6364	1.0000	0.7778

The per-class results show that the model performs strongly on the most important and most represented classes. The little owl class achieved an F1-score of 0.96, with both high precision and high recall. This is important because the owl is present in many relevant frames and forms the visual context in which prey animals are detected.
Several prey classes also achieved strong results. vogel, nachtvlinder, larve, meikever and regenworm all reached F1-scores above 0.90. This shows that the model is not only detecting the owl but is also able to recognize multiple prey categories.

The recall values are high for most classes. This is useful for the project, because missing prey detections would be more problematic than sending a limited number of additional detections to the human validation step. For example, nachtvlinder, vogel, larve, meikever, regenworm, veldmuis and bosmuis all achieved recall values above 0.90 or close to it.
Some classes have lower precision or F1-scores, mainly where the number of available evaluation examples is small. For example, loopkever has only seven ground-truth examples in the evaluation set. The model detected all seven of them, resulting in a recall of 1.00, but it also produced four false positives, which lowered the precision to 0.64. This result is still useful, but it should be interpreted together with the small ground-truth examples.

Overall, the per-class evaluation supports the general model results. The selected YOLOv12 nano model achieved strong performance in the owl class and in several prey categories. The lower scores are mainly linked to classes with fewer evaluation examples. This confirms that the model is suitable as a practical detection baseline for the current project, while future annotation work can focus on strengthening the less represented final classes.

Video Processing Pipeline

The trained YOLOv12 nano model was integrated into a video-processing pipeline that automatically identifies frames containing both an owl and a prey animal. The pipeline operates on MP4 video recordings and is intended to reduce the amount of footage that must be reviewed manually.

During inference (the process of applying a trained model to new data), videos are not processed frame by frame individually. Instead, frames are extracted from the video and grouped into batches before being sent to the model. The batch size (the number of frames processed simultaneously) can be adjusted depending on the available hardware. Larger batch sizes generally increase processing speed when sufficient GPU memory is available.

For each frame, the model predicts object locations, object classes, and confidence scores. A confidence score represents how certain the model is that a detected object belongs to a specific class and ranges from 0 to 1.

A frame is considered relevant only when:

- an owl is detected with a confidence score of at least 0.25.
- a prey animal is detected with a confidence score of at least 0.16.
- both the owl and the prey animal are present in the same frame.

These threshold values were selected after testing different configurations on the validation dataset. A higher threshold removed too many true prey detections, while a lower threshold produced a large number of false positives. The selected values provided a practical balance between detecting as many prey events as possible and limiting the number of incorrect detections that must be reviewed manually.

Frames that do not satisfy these conditions are discarded. The remaining frames are stored as candidate prey-delivery events for further ranking and review.

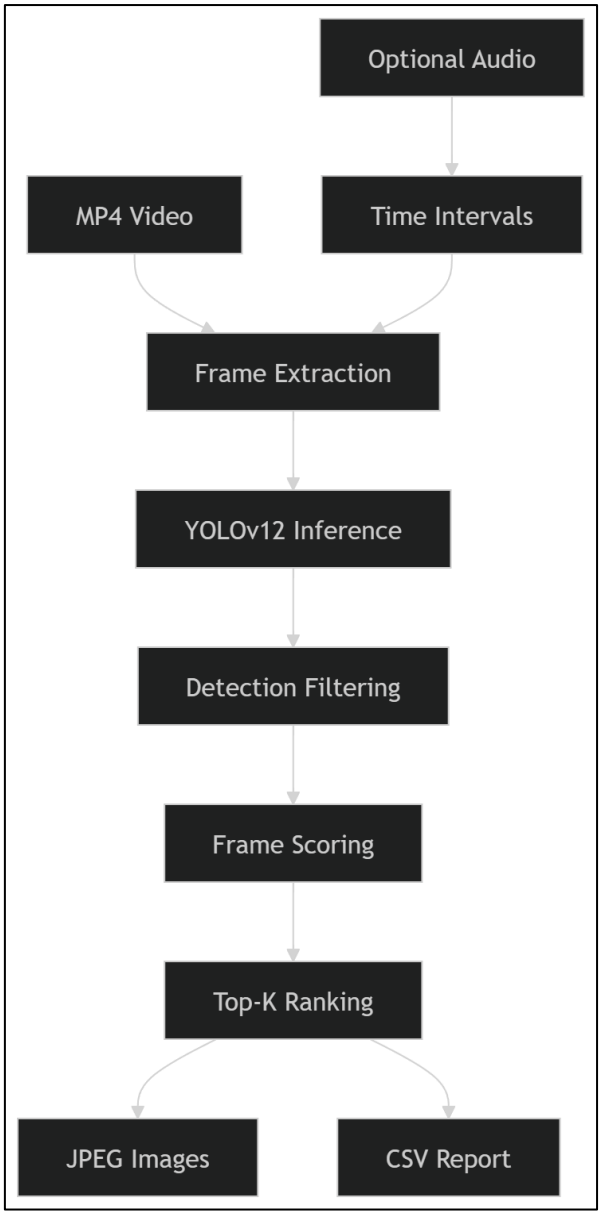


Figure 9, Workflow of pipeline containing the detection model

This diagram shows the overall processing workflow used by the pipeline. The final output consists of JPEG images and CSV files containing prediction information.

Audio-Guided Processing

In addition to the standard workflow, the pipeline supports an optional audio-guided mode. This mode was developed to reduce the number of video frames that need to be analyzed by the object-detection model.

When enabled, the audio track is extracted from the MP4 recording and converted into a WAV file. The audio signal is then analyzed using a sliding window of 100 milliseconds. For each window, the Root Mean Square (RMS) loudness is calculated and converted to decibels (dB, a logarithmic unit used to measure sound levels).

Audio segments that exceed a configurable loudness threshold are considered potential events. Short noise events are removed, while the remaining intervals are extended slightly before and after the detected sound. Overlapping intervals are merged together.

Only frames located inside these selected time intervals are processed by the YOLO model. All later processing steps remain identical to the standard pipeline.

If no sufficiently loud audio events are found, the pipeline terminates early without loading the YOLO model or processing video frames. This can significantly reduce processing time when recordings contain little or no activity.

The audio analysis is used only as a pre-filter to identify potentially interesting parts of the video. It does not influence the object-detection results or confidence scores. Final decisions are still made entirely by the visual detection model.

Top-K Frame Selection

Many videos contain multiple consecutive frames showing the same prey-delivery event. Saving all of these frames would increase storage requirements and create unnecessary work during manual validation. To address this issue, the pipeline includes a ranking mechanism that selects only the most informative frames.

After object detection, only frames containing both an owl and a prey animal are retained. Each frame receives a frame score based on the lowest confidence value among the required object groups. For example, if the owl confidence is 0.91 and the prey confidence is 0.63, the frame score becomes 0.63.

Using the minimum confidence ensures that a frame receives a high score only when both required detections are reliable. A frame with a very confident owl detection, but an uncertain prey detection should not be ranked highly.

Candidate frames are sorted according to this score. The pipeline then selects the highest-ranked frames while maintaining a minimum temporal distance between selected frames. This prevents the final results from containing many nearly identical images from the same event.

The number of saved frames is controlled by the *top-k* parameter. For example, a value of five saves only the five highest-ranked candidate frames. Alternatively, the value "*all"* can be used to save every valid detection.

The selected frames are saved together with their timestamps, confidence scores and detected classes. These outputs are intended for human-in-the-loop validation, where researchers or volunteers can quickly review the most relevant prey-detection events instead of manually inspecting entire video recordings.

7. In-The-Loop validation

The general tool [25] developed for STONE by Hogeschool Leiden already contains an interface which enables a user to replace and delete classifications from frames. The focus of the interface was to ‘keep it simple’ and provide insight into the data. This meant that a user must be able to quickly understand the tool and not feel overwhelmed by certain aspects of the tool. The last thing this implies is that the interface must show statistics of the data. The following image is what the interface looks like the moment a user logs in. Already statistics like the number of videos, cameras and locations are shown among other things.

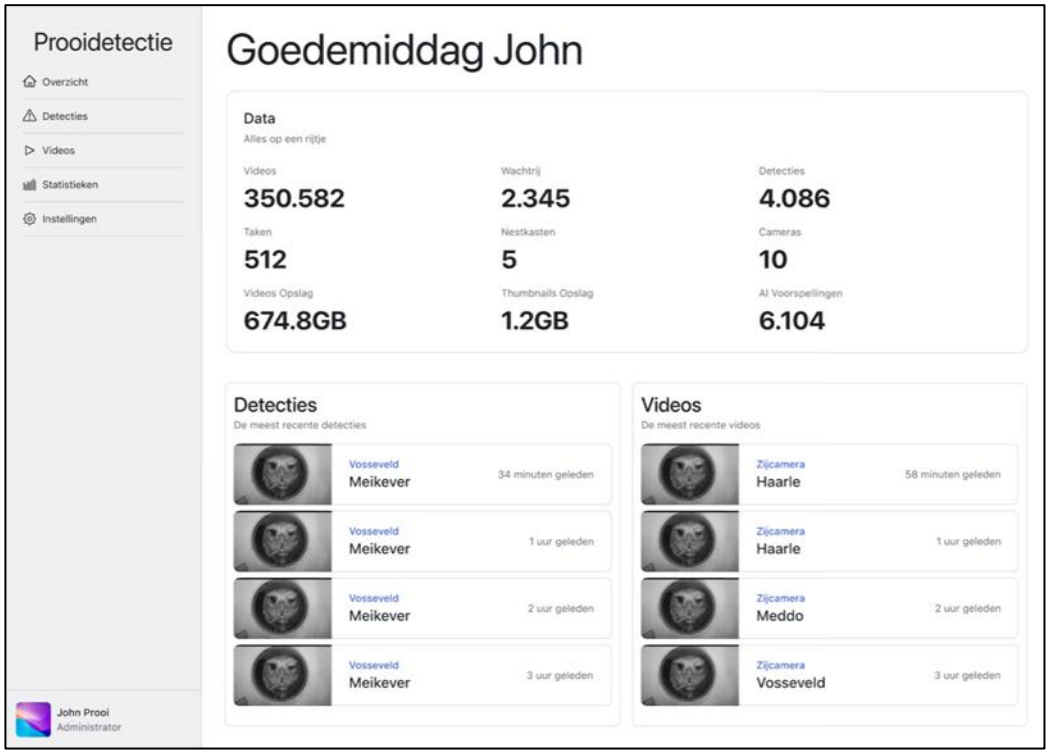


Figure 10, Concept design of the interface for in-the-loop validation

What is important for this assignment is the section where a user can change the classification of prey within a frame. The user is shown the frame with the classification of the prey. A dropdown menu contains the species a user can choose from if the model:

- Wrongly classified the prey animal
- Did not produce a single sufficient confidence score for any species

The dropdown menu must contain at least the same species shown in Species to classify, because the species present there are present in at least 5 areas where nestboxes have been placed. Note that the interface that is shown is part of the concept design. As of the time this report is written (31-05-2026), DataLab Hogeschool Leiden plans on developing and delivering this interface to STONE, but that has not been realized yet.

8. Data Ethics Decision Aid

The DEDA [26] (Data Ethics Decision Aid) is a toolkit to help identify ethical and practical issues of data-driven and AI projects. DEDA was introduced in 2018 and has since been used by organizations to pinpoint areas of concern and promote accountability to stakeholders and the public.

The DEDA was filled in for this project to achieve a better view of how the project and product affect the outside world. Then again, the project also focusses on applying AI responsibly in an ecological research setting, as was stated in the first chapter. The DEDA is made up of a total of 50 questions and four parts. The first part acts as a general chapter, including project information. The second part is the most technical in the way that it asks 18 data and algorithm related questions. The third part consists of the greatest number of questions, and they are more general about the stakeholders and the project in a broader sense. The fourth and final part is the conclusion and asks 4 questions about the insight gained from filling in the previous questions.

Action points are the product of the DEDA after everything has been filled in. In case of this project, these actions are related to anonymization, responsibility, transparency, and privacy. The exact actions are:

1. How is the video data transformed to conceal personal information?
2. What are the responsibilities of DataLab Hogeschool Leiden towards STONE?
3. Do citizens get the opportunity to raise objections towards the results of the project?
4. What laws, regulations, and guidelines apply to the project?

Point 1 is a data transformation step of when video data is collected and stored in an Azure blob. The data could be transformed to possibly conceal personal information, but that information does not have to be known by this project group. Point 2 is about the full responsibilities of DataLab Hogeschool Leiden towards STONE. That an application must be delivered is known, but the full scope and all responsibilities are unknown. Point 3 was uncovered by filling in the DEDA, but there is little to no chance that citizens from outside of the project would want to raise objections, because the project and its results will most likely not affect them. Point 4 shows that the exact laws and regulations that apply to the project are not known by the project group. Guidelines have been set up by Hogeschool Inholland Amsterdam and DataLab Hogeschool Leiden that have been upheld during the project.

9. Conclusion

This chapter answers the questions asked in chapter 3. The sub-questions will be answered in the order that can be found in that chapter. The main question is answered last in this chapter.

To start, the data was acquired in the form of MP4 videos. The data varied in the way that Vosseveld had recorded tenths of thousands of less videos compared to the other locations. In addition, the camera angle that was placed inside the nestbox ('ch3') is also overrepresented in the data compared to the other camera angles. The data stabilizes in its duration in seconds with only some outliers of video recordings being longer than ten minutes. What contributes the most towards the stability of the data is the distribution of the hours of the day a recording starts and the recordings per month of the breeding season (April, May, and June).

The frames containing owls and prey were easily extracted from recordings from 2024 because it was labelled. Recordings from 2025 required more work. To summarize, only valid videos were processed for every fourth frame that was checked for sufficient motion and sharpness, to capture moments where owls were bringing prey to the nest. This left the project group with frames mostly containing owls and their prey. This was annotated with RoboFlow, CVAT, and checked with a custom Python interface. The annotated data from the previous project also needed to be re-annotated and checked. The data was split into a train set of 3800 annotations and a test set of 579 annotations.

Three candidate models have been tested for the project. These are YOLOv12 nano, RT-DETR and YOLOv10 nano. The first model, YOLOv12 nano, portrayed the highest metric values, except for recall, and was thus chosen to be trained further. Different image sizes were tested, but greater images sizes, greater than 640x640, resulted in worse metric values and longer processing/training time. The final YOLOv12 nano model achieved the best performance values after having been trained for 89 epochs.

Volunteers and/or researchers of STONE will have access to an interface in the future that is still being developed for them by DataLab, Hogeschool Leiden. The interface focusses on being easy to understand and use by not overwhelming the user with details. In addition, the interface must show certain insightful statistics and provide different tabs, so a user can view the videos and detections. It is from the detections tab where a user can assign, modify, and delete a label given to a prey animal by the detection model.

Main question:
To answer the main research question: a YOLOv12 nano object detector, trained on around 3,800 annotated frames from the nest-box cameras and used together with the validation interface that was designed at Hogeschool Leiden, can classify the prey that owls bring into their nest accurately. On the held-out evaluation set, the model reached an mAP@0.5 of 0.96 and F1 scores above 0.90 for most of the prey species. The validation step also makes sure that STONE volunteers stay in control of every label that ends up in the final dataset, which means the system supports the researchers without replacing them.

Sand vs Clay:
The Context at the start of this report mentioned a finding from STONE's earlier research that owlets that developed in sandy-soil areas are in better physical shape than owlets raised in clay-soil areas. The explanation for this most likely has to do with what kind of prey is available in each area, and so with what the owls eat.
Our project did not try to answer that ecological question directly. The assignment that was given by STONE and DataLab Hogeschool Leiden was to build a system that can automatically detect and identify the prey in nest-box footage, so that the researchers at STONE can spend their time studying the data rather than collecting it by hand.

The clay-versus-sand comparison is exactly the kind of question that becomes easier to answer with what has been built. The trained YOLOv12 nano model can go through thousands of nest-box recordings and suggest what prey is in each frame. The validation interface lets STONE volunteers quickly check those labels and correct them if needed. The result is a cleaner dataset of prey events per location, which the STONE researchers can then group by soil type and use to compare what owls in sandy areas bring into the nest with what owls in clay areas bring in. Thus, the model is not doing the clay-versus-sand analysis itself, but it is a tool that makes that analysis easier to do later. This also fits with what the assignment asked to deliver: a system that supports the researchers, not one that replaces them.

10. Discussion

The detection model is able to capture and label the prey well in the test set. Even the species "Loopkever" was captured relatively well by the model while only having been trained on 84 annotations of the species. That the pipeline produces three frames from which a volunteer of STONE can decide what the prey animal is providing insight into the detection. The drawback of this is that it must be integrated with the interface of the application for in-the-loop validation. This will be done by DataLab, Hogeschool Leiden after the conclusion of the current project that this report encapsulates. It is expected that the developers will contact the project group from Hogeschool Inholland Amsterdam for aid.

A short coming from the data annotation is the number of species present in the annotations. A minimum of 14 species/animal groups, including the little owl, are present in areas where nestboxes have been placed. However, the project only managed to annotate enough to accurately detect the little owl and 11 species of prey animal. This is the result of annotations needed to be done manually and labelling the unlabeled 2025 data was time consuming. A suggestion for making this step easier is for STONE to store videos/pictures of owls bringing in prey in a prey specific folder. This allows for a new team to easily label and annotate a frame, because the prey species is already given.

The next iteration should prioritize enhancing the current model. To elaborate on that, increase the number of species the model can predict. The current model is good at detecting owls and prey, but only for the ones it has been trained on, and it does make a few mistakes. Luckily, in-the-loop validation counteracts that issue by letting the people at STONE correct the label. A more desirable situation could be that the frame already has the right label, and a volunteer/researcher can move on to the next frame/detection.

11. References

[1] R. Van Harxen, P. Stroeken, T. Boudewijn, and Van Turnhout, “50 jaar reproductie van de steenuil in Nederland,” 2023.

[2] R. Caballar and C. Stryker, “What is computer vision?,” https://www.ibm.com/think/topics/computer-vision.

[3] D. Bergmann, “What is machine learning?,” https://www.ibm.com/think/topics/machine-learning.

[4] Ben. Baarda and David. Hidajattoellah, *Research. This is it! : guidelines how to design, perform and evaluate quantitative and qualitative research*. Noordhoff Uitgevers, 2014.

[5] LF Charities, “Jupyter,” https://jupyter.org/.

[6] F. Chollet and M. Watson, *Deep learning with Python - Chapter 12: Object detection*. 2025. Accessed: Mar. 22, 2026. [Online]. Available: https://deeplearningwithpython.io/chapters/chapter12_object-detection/

[7] Roboflow, “AI that sees and understands the physical world,” https://roboflow.com/.

[8] CVAT.ai Corporation, “Open Data Annotation Platform,” https://www.cvat.ai/.

[9] U. A. Keerio, “Why Deep Learning is so called Data-Hungry.,” https://www.linkedin.com/pulse/why-deep-learning-so-called-data-hungry-umed-ali-keerio/.

[10] OpenCV team, “About,” https://opencv.org/about/.

[11] GeeksforGeeks, “How to Check for Blurry Images in Your Dataset Using the Laplacian Method,” https://www.geeksforgeeks.org/computer-vision/how-to-check-for-blurry-images-in-your-dataset-using-the-laplacian-method/.

[12] Heartex Inc, “Open Source Data Labeling Tool,” http://labelstud.io.s3-website-us-east-1.amazonaws.com/.

[13] R. Van Harxen, “Voedselonderzoek Steenuil op klei en zand,” Nov. 2024.

[14] Ultralytics, “Data Collection and Annotation Strategies for Computer Vision,” https://docs.ultralytics.com/guides/data-collection-and-annotation#how-can-i-ensure-high-consistency-and-accuracy-in-data-annotation.

[15] P. Jiang, D. Ergu, F. Liu, Y. Cai, and B. Ma, “A Review of Yolo Algorithm Developments,” *Procedia Comput. Sci.*, vol. 199, pp. 1066–1073, 2022.

[16] Ultralytics, “YOLO12: Attention-Centric Object Detection,” https://docs.ultralytics.com/models/yolo12.

[17] IBM, “What is overfitting?,” https://www.ibm.com/think/topics/overfitting.

[18] R. Padilla, S. L. Netto, and E. A. B. da Silva, “A Survey on Performance Metrics for Object-Detection Algorithms,” in *2020 International Conference on Systems, Signals and Image Processing (IWSSIP)*, 2020, pp. 237–242. doi: 10.1109/IWSSIP48289.2020.9145130.

[19] P. Kashyap, “Understanding Precision, Recall, and F1 Score Metrics,” https://medium.com/@piyushkashyap045/understanding-precision-recall-and-f1-score-metrics-ea219b908093.

[20] J. Hui, “mAP (mean Average Precision) for Object Detection,” https://jonathan-hui.medium.com/map-mean-average-precision-for-object-detection-45c121a31173.

[21] Ultralytics Inc, “Model Weights,” https://www.ultralytics.com/glossary/model-weights.

[22] Ultralytics Inc, “Epoch,” https://www.ultralytics.com/glossary/epoch.

[23] CloudFactory Limited, “Patience,” https://wiki.cloudfactory.com/docs/mp-wiki/training-parameters/patience.
[24] Dremio, “Grid Search,” https://www.dremio.com/wiki/grid-search/.
[25] T. D. Justification. 2025 Hogeschool Leiden, “Design Justification,” 2025.
[26] Utrecht University, “Data Ethics Decision Aid (DEDA),” https://deda.dataschool.nl/en/.